

RAPPORT TECHNIQUE
Installation et Configuration de Prometheus, Grafana
et OpenTelemetry



Prometheus



Grafana



OpenTelemetry

Réaliser par :

Amadou SY

Etudiant en M1 SSIM (Sécurité des Systèmes d'Information et
Monétiques)

Ecole de formation : Groupe IZI

Email : sidiahmadsy@gmail.com

Sous la supervision de :

Monsieur LÔ

Table des matières

1. Introduction.....	2
1.1. Problématique	2
1.2. Présentation du stack de supervision.....	3
1.3. Environnement matériel et réseau.....	4
1.3.1. Role de chaque composant	4
2. Installation de Prometheus	4
2.1. Télécharger Prometheus depuis le site officiel : https://prometheus.io/download	4
2.2. Extraction du fichier compressé :	4
2.3. Créer un répertoire pour Prometheus et déplacez-y les fichiers extraits :	4
2.4. Vérifier l'installation	5
2.5. Préparer un utilisateur système dédié à Prometheus et définir les permissions nécessaires.....	5
2.6. Création du service systemd.....	5
2.7. Activation et démarrage de Prometheus	5
2.8. Interface web de Prometheus	6
3. Installation et configuration de Node Exporter (Client Ubuntu : 192.168.1.200)	6
3.1. Télécharger Node Exporter depuis : https://prometheus.io/download/#node_exporter	6
3.2. Installer Node Exporter :	6
3.3. Créer un utilisateur système dédié à Node Exporter , utilisée pour des raisons de sécurité.	6
3.4. Créer un service systemd pour lancer Node Exporter au démarrage :	6
3.5. Démarrage et activation de Node Exporter :.....	7
3.6. Accès à l'interface web du Node Exporter :	7
4. Installation et configuration de Node Exporter (Client Windows : 192.168.1.223)	7
5. Configuration de Prometheus pour surveiller les nœuds clients	9
5.1. Modifier le fichier <code>/etc/prometheus/prometheus.yml</code>	9
5.2. Ajouter les deux agents node exporter	9
5.3. Redémarrer Prometheus :	10
5.4. Accéder l'interface web de Prometheus pour vérifier la présence des deux agents	10
6. Installation de OpenTelemetry Collector.....	10
6.1. Télécharger le paquet depuis le site : https://opentelemetry.io/docs/collector/getting-started/	10
6.2. Installer le paquet OpentTelemetry Collector	10
6.3. Redémarrage et activation d'OpenTelemetry Collector	11

7. Installation et configuration de Grafana.....	11
7.1. Ajout de clé CPG :	11
7.2. Ajout de dépôt de Grafana :	11
7.3. Installer Grafana	11
7.4. Lancer Grafana :	11
7.5. Accès à l'interface web	11
8. Ajouter une source de donnée dans Grafana.....	12
9. Ajouter un dashboard dans Grafana	13
10. Conclusion	18

1. Introduction

Dans un environnement informatique moderne, la supervision et l'observation des systèmes sont essentielles pour garantir la disponibilité, la performance et la fiabilité des services.

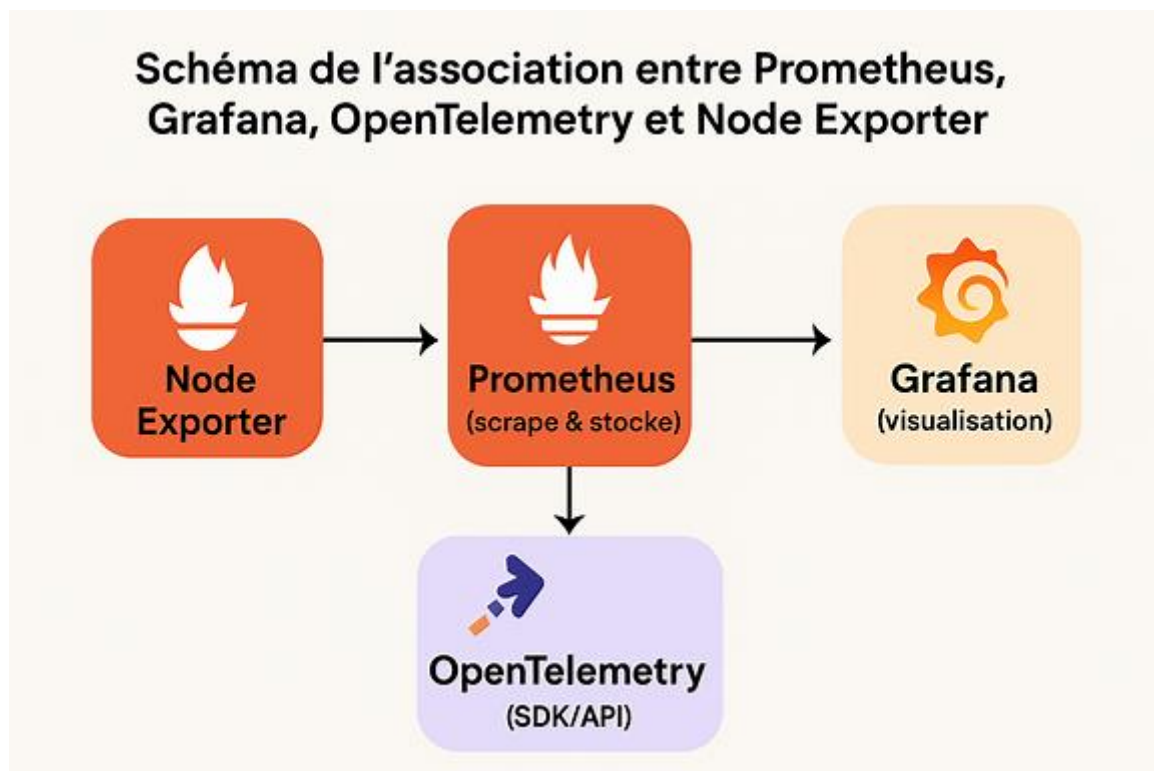
La stack composée de **Prometheus**, **Node Exporter**, **OpenTelemetry** et **Grafana** offre une solution complète pour collecter, stocker, analyser et visualiser les métriques système et applicatives.

Ce manuel technique a pour objectif de fournir un guide pas-à-pas pour l'installation, la configuration et la mise en œuvre de ces outils sur un environnement Linux, tout en assurant une structure claire et reproductible.

1.1. Problématique

Comment mettre en place une solution de supervision moderne, unifiée et extensible, capable de surveiller efficacement des environnements hétérogènes (Linux et Windows), tout en garantissant la disponibilité, la performance et la fiabilité des services informatiques ?

1.2. Présentation du stack de supervision



La solution présentée repose sur quatre composants principaux :

1. Prometheus

- Rôle : Collecte et stocke les métriques dans une base de données de séries temporelles.
- Points forts : Langage de requête *PromQL*, haute performance, intégration facile avec de multiples exportateurs.

2. Node Exporter

- Rôle : Expose les métriques matérielles et système (CPU, mémoire, disque, réseau) d'un nœud.
- Points forts : Légèreté, facilité de déploiement, compatibilité native avec Prometheus.

3. OpenTelemetry Collector

- Rôle : Standardise, transforme et envoie les données de télémétrie (logs, traces, métriques) vers des backends tels que Prometheus ou Grafana.
- Points forts : Compatibilité multi-protocoles, flexibilité, interopérabilité.

4. Grafana

- Rôle : Visualisation et analyse des métriques sous forme de tableaux de bord interactifs.
- Points forts : Interface intuitive, personnalisation avancée, intégrations multiples.

1.3. Environnement matériel et réseau

Nom / Rôle	Adresse IP	Système d'exploitation	Logiciels installés	Ports utilisés / URL d'accès
myserver3 (machine principale)	192.168.1.100	Ubuntu 22.04 LTS	Prometheus, OpenTelemetry Collector, Grafana	Prometheus → :9090 → http://192.168.1.100:9090 Grafana → :3000 → http://192.168.1.100:3000
Client-Virtualbox	192.168.1.200	Ubuntu 22.04 LTS	Node Exporter	Node Exporter → :9100 → http://192.168.1.200:9100/metrics
Client – VirtualBox	192.168.1.223	Windows 10	Node Exporter	Node Exporter → :9182 → http://192.168.1.223:9182/metrics

1.3.1. Role de chaque composant

- myserver3 :
 - Il joue le rôle de **serveur maître** qui centralise la collecte, le traitement et l’affichage des données de tous les clients supervisés.
- Client Ubuntu :
 - Il fournit des **données de performance système Linux** que Prometheus collecte pour analyse.
- Client Windows :
 - Il permet de **surveiller un poste ou serveur Windows** au même titre que les machines Linux, pour une supervision homogène.

2. Installation de Prometheus

2.1. Télécharger Prometheus depuis le site officiel : <https://prometheus.io/download>

```
mistersy@myserver3:~$ wget https://github.com/prometheus/prometheus/releases/download/v2.55.0/prometheus-2.55.0.linux-amd64.tar.gz
```

2.2. Extraction du fichier compressé :

```
mistersy@myserver3:~$ tar -xvf prometheus-*.tar.gz
```

2.3. Créer un répertoire pour Prometheus et déplacez-y les fichiers extraits :

```
root@myserver3:/home/mistersy# mkdir /etc/prometheus  
mkdir /var/lib/prometheus
```

```
root@myserver3:/home/mistersy# mv prometheus-2.55.0.linux-amd64/prometheus /usr/local/bin/
mv prometheus-2.55.0.linux-amd64/promtool /usr/local/bin/
mv prometheus-2.55.0.linux-amd64/prometheus.yml /etc/prometheus/
```

2.4. Vérifier l'installation

```
root@myserver3:/home/mistersy# prometheus --version
prometheus, version 2.55.0 (branch: HEAD, revision: 91d80252c3e528728b0f88d254dd720f6be07cb8)
  build user:      root@9fad779131cc
  build date:      20241022-13:47:22
  go version:      go1.23.2
  platform:        linux/amd64
  tags:            netgo,builtinassets,stringlabels
```

2.5. Préparer un utilisateur système dédié à Prometheus et définir les permissions nécessaires.

```
root@myserver3:/home/mistersy# useradd --no-create-home --shell /bin/false prometheus
chown prometheus:prometheus /etc/prometheus/prometheus.yml
chown -R prometheus:prometheus /var/lib/prometheus
```

2.6. Création du service systemd

```
root@myserver3:/home/mistersy# nano /etc/systemd/system/prometheus.service
```

Contenu à ajouter dans le fichier `/etc/systemd/system/prometheus.service` :

```
root@myserver3: /home/mistersy
GNU nano 6.2 /etc/systemd/system/prometheus.service *
[Unit]
Description=Prometheus
After=network.target

[Service]
User=prometheus
Group=prometheus
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/var/lib/prometheus/

[Install]
WantedBy=multi-user.target
```

2.7. Activation et démarrage de Prometheus

```
root@myserver3:/home/mistersy# systemctl daemon-reload
```

```
root@myserver3:/home/mistersy# systemctl start prometheus
systemctl status prometheus
● prometheus.service - Prometheus
   Loaded: loaded (/etc/systemd/system/prometheus.service; disabled; vendor preset: enabled)
   Active: active (running) since Thu 2025-05-15 09:45:52 GMT; 41ms ago
     Main PID: 7552 (prometheus)
        Tasks: 4 (limit: 4606)
       Memory: 2.0M
          CPU: 7ms
      CGroup: /system.slice/prometheus.service
              └─7552 /usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.p

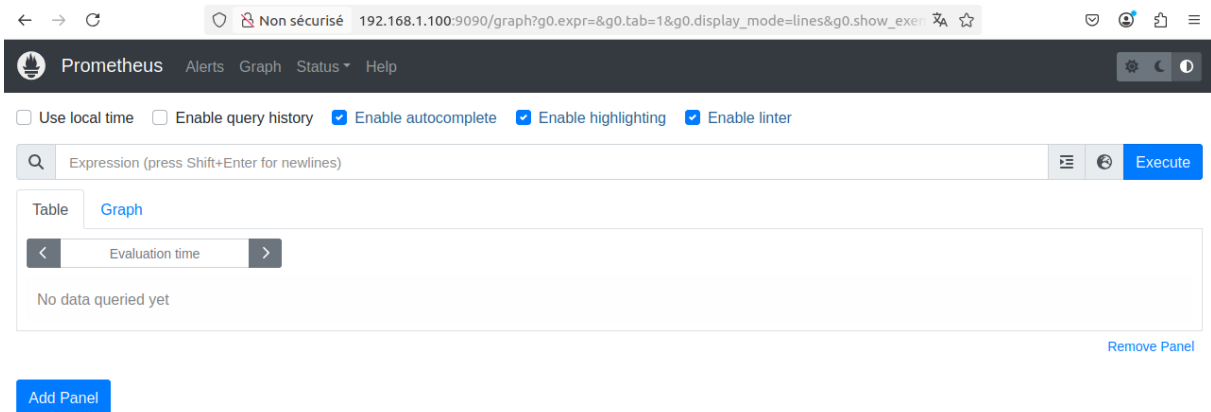
ne 15 09:45:52 myserver3 systemd[1]: Started Prometheus.
lines 1-11/11 (END)
```

```
root@myserver3:/home/mistersy# ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.100 netmask 255.255.255.0 broadcast 192.168.1.255
```

2.8. Interface web de Prometheus

Par défaut, Prometheus sera accessible via le navigateur à l'adresse :

http://<IP_du_serveur>:9090



3. Installation et configuration de Node Exporter (Client Ubuntu : 192.168.1.200)

3.1. Télécharger Node Exporter depuis :

https://prometheus.io/download/#node_exporter

```
root@client-VirtualBox:/home/client# wget https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
```

3.2. Installer Node Exporter :

```
root@client-VirtualBox:/home/client# tar -xvf node_exporter-1.8.2.linux-amd64.tar.gz
```

```
root@client-VirtualBox:/home/client# mv node_exporter-1.8.2.linux-amd64/node_exporter /usr/local/bin/
```

3.3. Créer un utilisateur système dédié à Node Exporter, utilisée pour des raisons de sécurité.

```
root@client-VirtualBox:/home/client# useradd --no-create-home --shell /bin/false node_exporter
```

3.4. Créer un service systemd pour lancer Node Exporter au démarrage :

```
root@client-VirtualBox:/home/client# nano /etc/systemd/system/node_exporter.service
```

Editer le fichier : `/etc/systemd/system/node_exporter.service`

```
root@client-VirtualBox: /home/client
GNU nano 6.2 /etc/systemd/system/node_exporter.service
[Unit]
Description=Node Exporter
After=network.target
[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter
[Install]
WantedBy=multi-user.target
```

3.5. Démarrage et activation de Node Exporter :

`sudo systemctl daemon-reload`

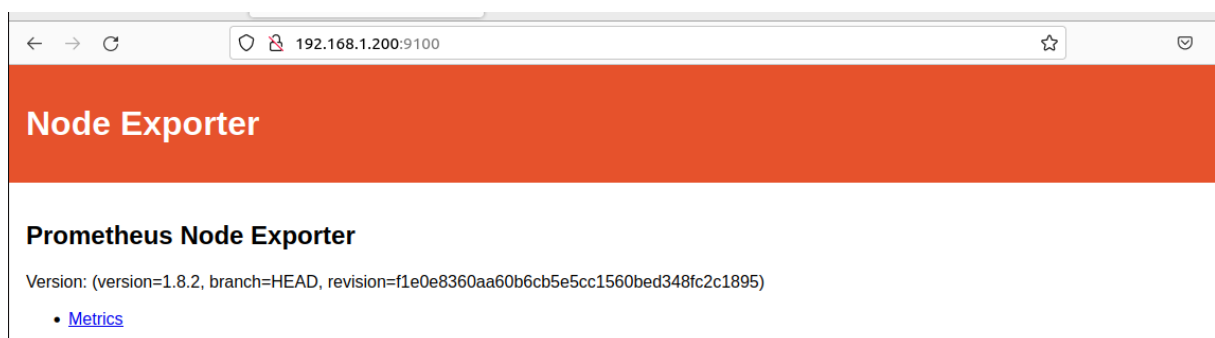
`sudo systemctl enable --now node_exporter`

```
root@client-VirtualBox:/home/client# systemctl enable node_exporter
Created symlink /etc/systemd/system/multi-user.target.wants/node_exporter.service → /etc/systemd/system/node_exporter.service.
root@client-VirtualBox:/home/client# systemctl start node_exporter
root@client-VirtualBox:/home/client# systemctl status node_exporter
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; vendor
   Active: active (running) since Fri 2025-05-16 09:26:29 GMT; 5s ago
     Main PID: 32406 (node_exporter)
       Tasks: 3 (limit: 1679)
      Memory: 2.1M
         CPU: 14ms
    CGroup: /system.slice/node_exporter.service
            └─32406 /usr/local/bin/node_exporter
```

3.6. Accès à l'interface web du Node Exporter :

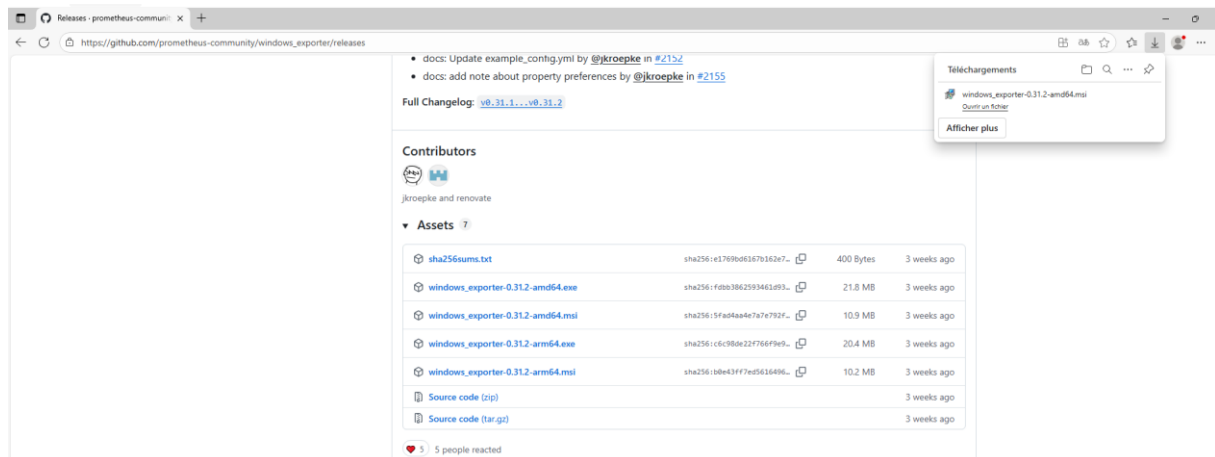
Node Exporter sera accessible via le navigateur à l'adresse :

`http://<IP_de_ton_serveur>:9100`

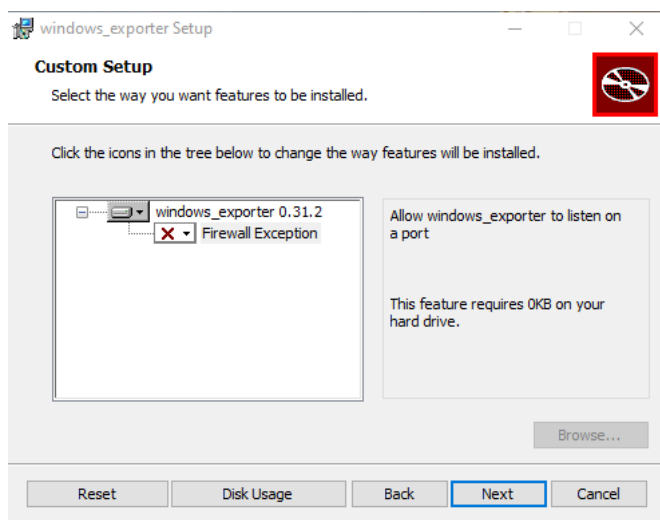


4. Installation et configuration de Node Exporter (Client Windows : 192.168.1.223)

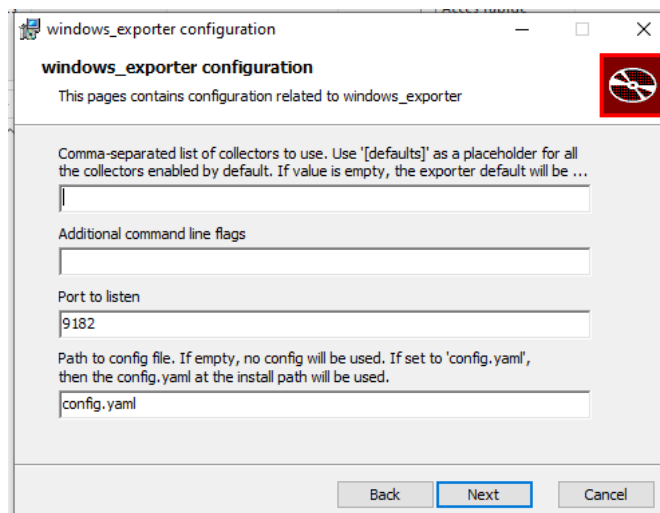
1. Télécharger exporter pour windows dans le site : https://github.com/prometheus-community/windows_exporter/releases



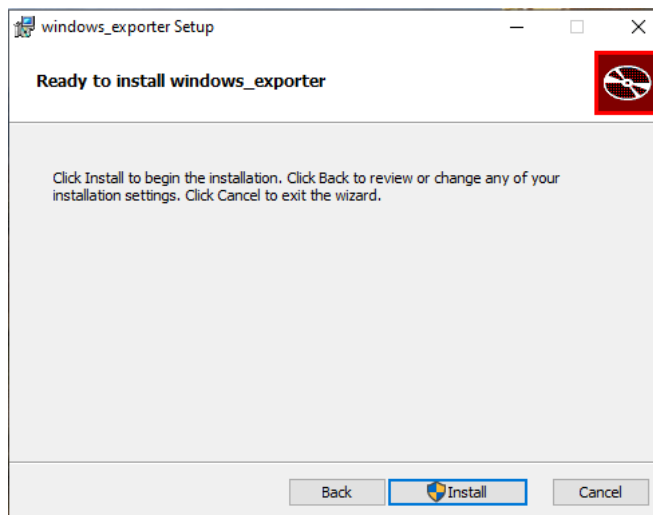
2. Lancer l'installateur



3. Choisir le port (par défaut le port 9182)



4. Finaliser avec le bouton « Install »



5. Ouvrir PowerShell en administrateur et autoriser le port :

```
PS C:\Windows\system32> New-NetFirewallRule -DisplayName "Autoriser Port 9182" -Direction Inbound -Protocol TCP -LocalPort 9182 -Action Allow

Name                           : {004bcbcd-667f-43aa-858f-90e8b8e87f3f}
DisplayName                     : Autoriser Port 9182
Description                     :
DisplayGroup                    :
Group                           :
Enabled                         : True
Profile                         : Any
Platform                       : {}
Direction                      : Inbound
Action                         : Allow
EdgeTraversalPolicy             : Block
LooseSourceMapping              : False
LocalOnlyMapping                : False
Owner                           :
PrimaryStatus                   : OK
Status                         : La règle a été analysée à partir de la banque. (65536)
EnforcementStatus               : NotApplicable
PolicyStoreSource               : PersistentStore
PolicyStoreSourceType           : Local
RemoteDynamicKeywordAddresses  :
PolicyAppId                     :
```

5. Configuration de Prometheus pour surveiller les nœuds clients

5.1. Modifier le fichier `/etc/prometheus/prometheus.yml`

```
root@myserver3:/home/mistersy# nano /etc/prometheus/prometheus.yml
```

5.2. Ajouter les deux agents node exporter

Dans targets ajouter les adresses ip des agents :

```
Fichier  Machine  Écran  Entrée  Périphériques  Aide
root@myserver3: /home/mistersy
# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "client_node"
    static_configs:
      - targets: ["192.168.1.26:9100", "192.168.1.223:9182"]
```

5.3. Redémarrer Prometheus :

```
root@myserver3:/home/mistersy# systemctl restart prometheus
```

5.4. Accéder l'interface web de Prometheus pour vérifier la présence des deux agents

← → ↻ local:3090/targets?search= Prometheus Alerts Graph Status Help

Targets

All scrape pools All Unhealthy Collapse All Filter by endpoint or labels Unknown Unhealthy Healthy

client_node (2/2 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.1.26:9100/metrics	UP	instance="192.168.1.26:9100" job="client_node" ▾	1.598s ago	41.056ms	
http://192.168.1.223:9182/metrics	UP	instance="192.168.1.223:9182" job="client_node" ▾	6.577s ago	5.423s	

prometheus (1/1 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus" ▾	1.203s ago	19.280ms	

6. Installation de OpenTelemetry Collector

6.1. Télécharger le paquet depuis le site :

<https://opentelemetry.io/docs/collector/getting-started/>

```
root@myserver3:/home/mistersy# wget https://github.com/open-telemetry/opentelemetry-collector-releases/releases/download/v0.108.0/otelcol_0.108.0_linux_amd64.deb
```

6.2. Installer le paquet OpentTelemetry Collector

```
root@myserver3:/home/mistersy# dpkg -i otelcol_0.108.0_linux_amd64.deb
```

```
root@myserver3:/home/mistersy# nano /etc/otelcol/config.yaml
```

6.3. Redémarrage et activation d'OpenTelemetry Collector

```
root@myserver3:/home/mistersy# systemctl restart otelcol
root@myserver3:/home/mistersy# systemctl status otelcol
● otelcol.service - OpenTelemetry Collector
   Loaded: loaded (/lib/systemd/system/otelcol.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2025-05-16 11:22:24 GMT; 7s ago
     Main PID: 53435 (otelcol)
        Tasks: 7 (limit: 4606)
       Memory: 15.2M
          CPU: 82ms
      CGroup: /system.slice/otelcol.service
              └─53435 /usr/bin/otelcol --config=/etc/otelcol/config.yaml
```

7. Installation et configuration de Grafana

```
root@myserver3:/home/mistersy# apt install -y software-properties-common apt-transport-https
```

7.1. Ajout de clé CPG :

```
root@myserver3:/home/mistersy# sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
```

7.2. Ajout de dépôt de Grafana :

```
root@myserver3:/home/mistersy# echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main
```

```
root@myserver3:/home/mistersy# sudo apt update
```

7.3. Installer Grafana

```
root@myserver3:/home/mistersy# sudo apt install grafana
```

7.4. Lancer Grafana :

```
root@myserver3:/home/mistersy# systemctl enable grafana-server
systemctl start grafana-server
```

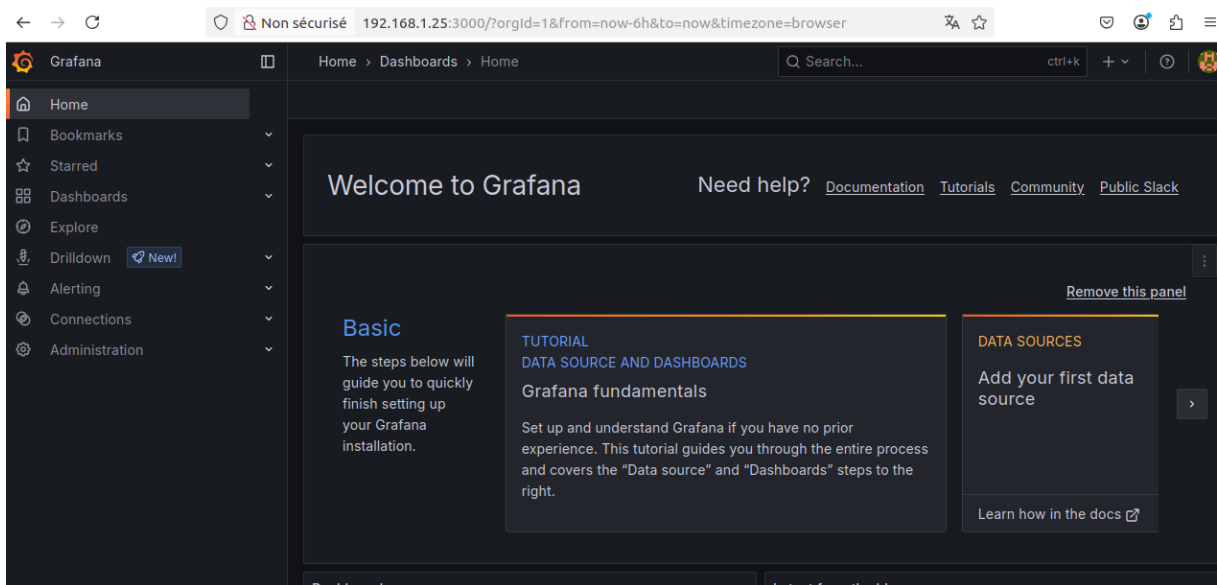
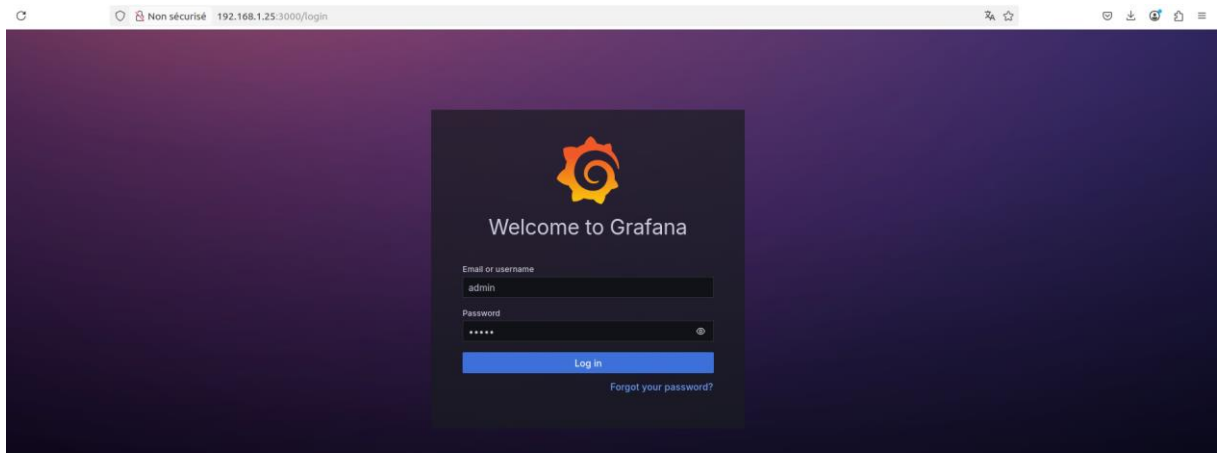
```
root@myserver3:/home/mistersy# systemctl status grafana-server
● grafana-server.service - Grafana instance
   Loaded: loaded (/lib/systemd/system/grafana-server.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2025-05-17 07:54:24 GMT; 1min 20s ago
     Docs: http://docs.grafana.org
    Main PID: 59173 (grafana)
         Tasks: 17 (limit: 4606)
        Memory: 172.5M
           CPU: 6.752s
       CGroup: /system.slice/grafana-server.service
               └─59173 /usr/share/grafana/bin/grafana-server --config=/etc/grafana/grafana.ini --pidfile=/var
```

7.5. Accès à l'interface web

Par défaut, Grafana sera accessible via le navigateur à l'adresse :

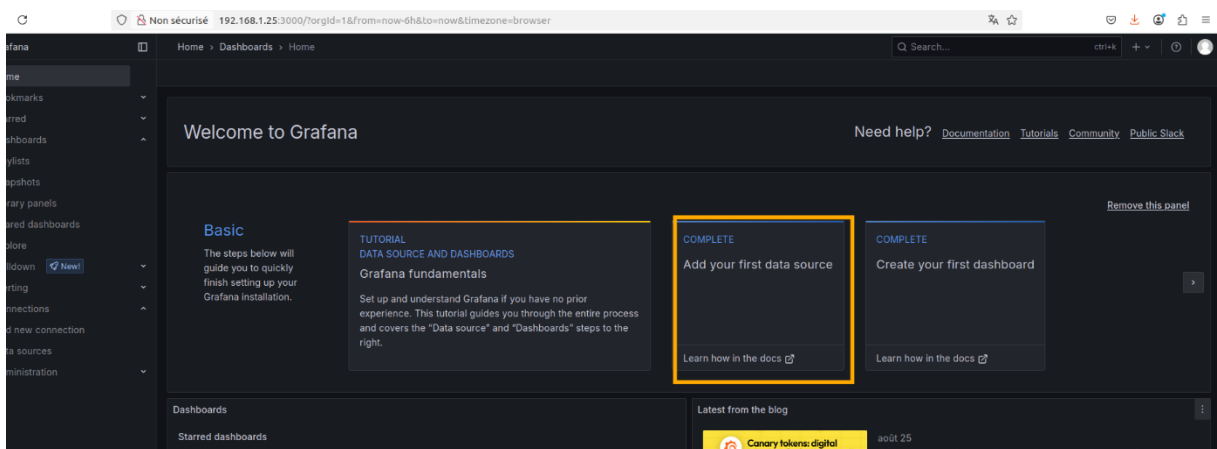
http://<IP_du_serveur>:3000

- **Identifiants par défaut :**
 - **Utilisateur :** admin
 - **Mot de passe :** admin (tu seras invité à le changer à la première connexion)

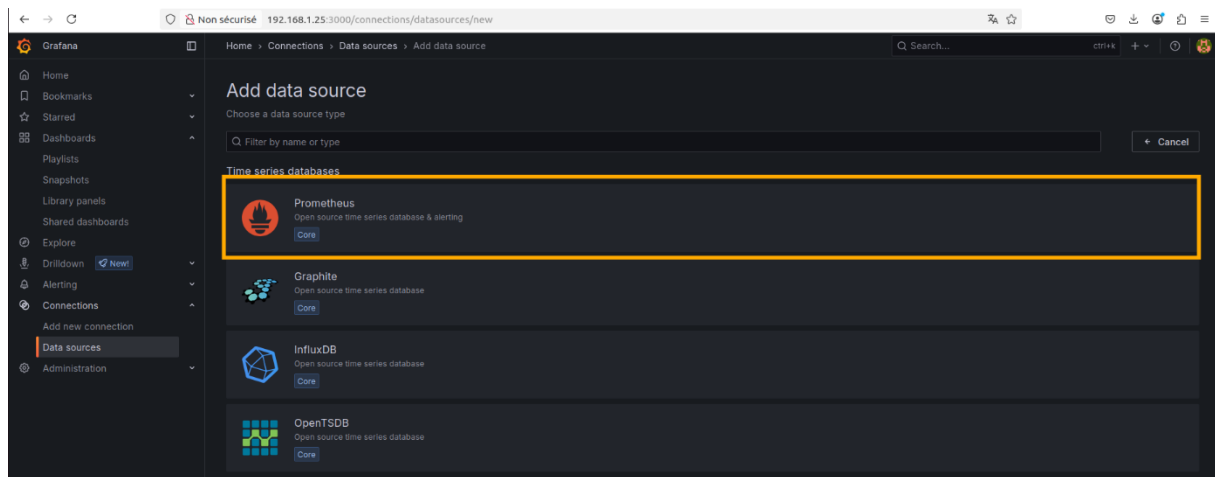


8. Ajouter une source de donnée dans Grafana

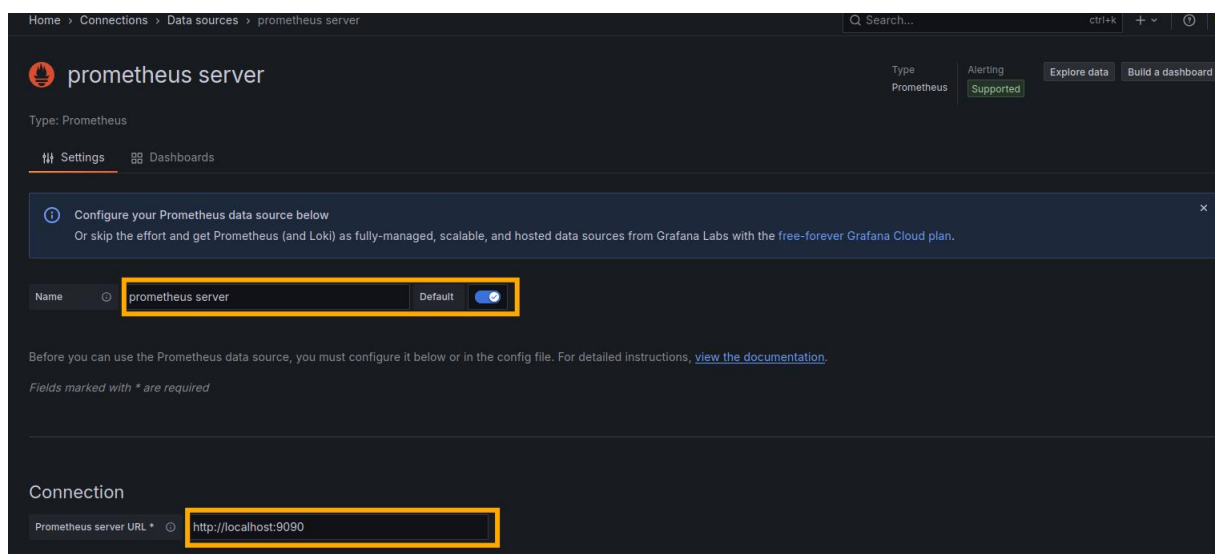
1. Se Connecter à Grafana (<http://tonserveur:3000>).
2. Aller dans **home > Dashboards**



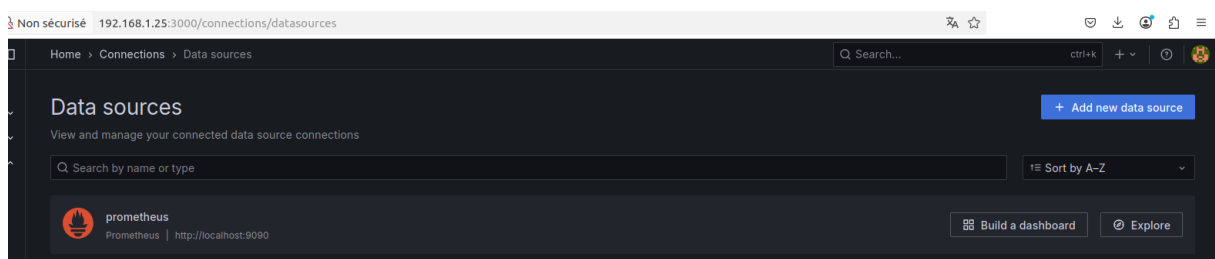
3. Choisir Prometheus



4. Renseigner les champs et confirmer

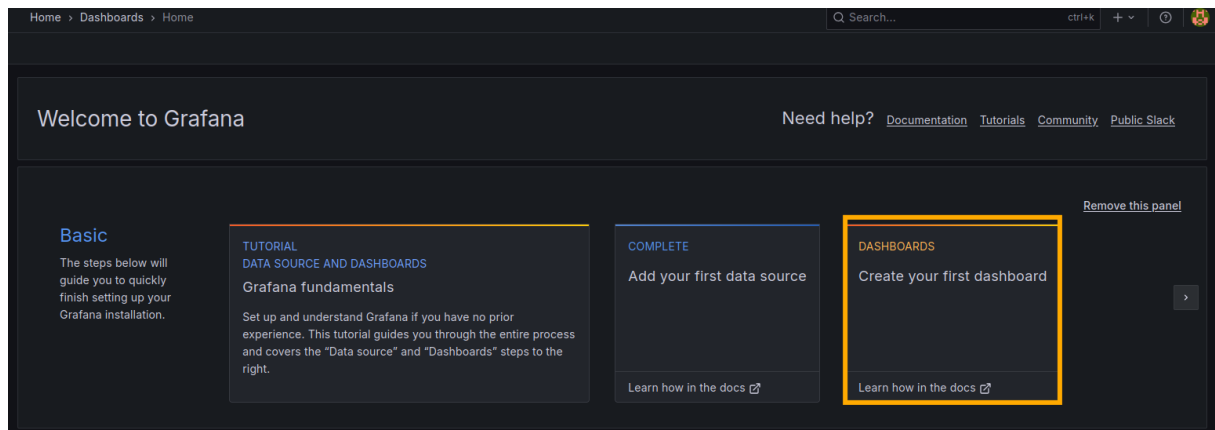


5. Vérifier que la source de donnée a bien été ajouté

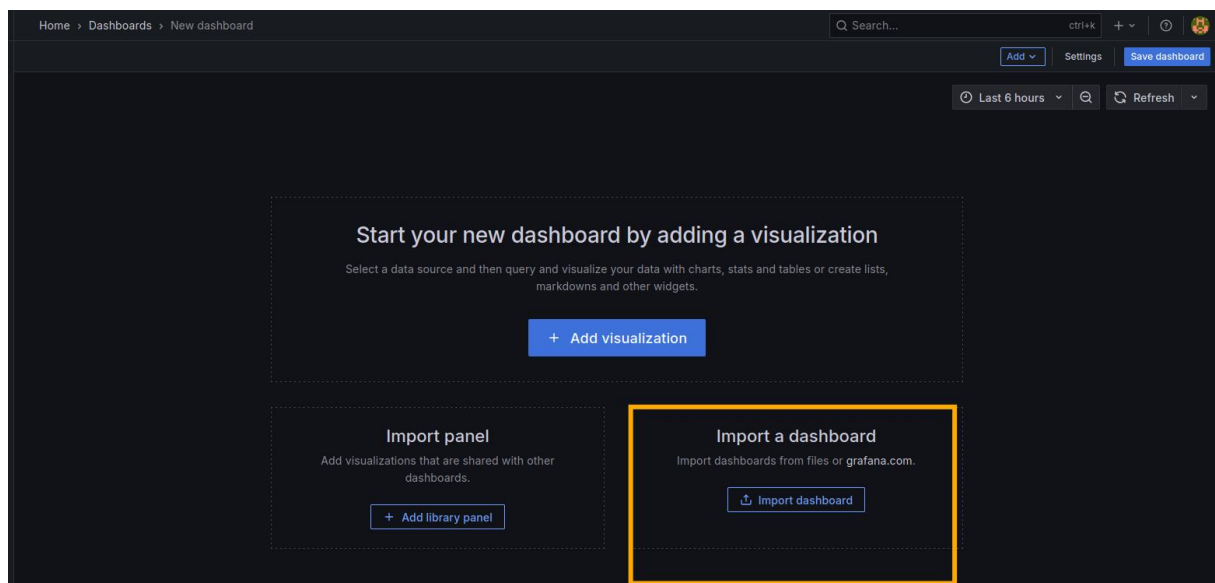


9. Ajouter un dashboard dans Grafana

1. Se Connecter à Grafana (<http://tonserveur:3000>).
2. Aller dans **Dashboards > Import**.



3. Importer un dashboard



4. Entrer un ID de dashboard populaire, par exemple :

- **1860** : Node Exporter Full

Import dashboard

Import dashboard from file or Grafana.com



Upload dashboard JSON file

Drag and drop here or click to browse
Accepted file types: .json, .txt

Find and Import dashboards for common applications at grafana.com/dashboards

1860

Load

Import via dashboard JSON model

```
{
  "title": "Example - Repeating Dictionary variables",
  "uid": "_OHnEoN4z",
  "panels": [...]
  ...
}
```

Load

Cancel

5. Sélectionner la **source de données Prometheus**.

Home > Dashboards > Import dashboard

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from Grafana.com

Published by rfmoz

Updated on 2025-06-12 20:06:17

Options

Name

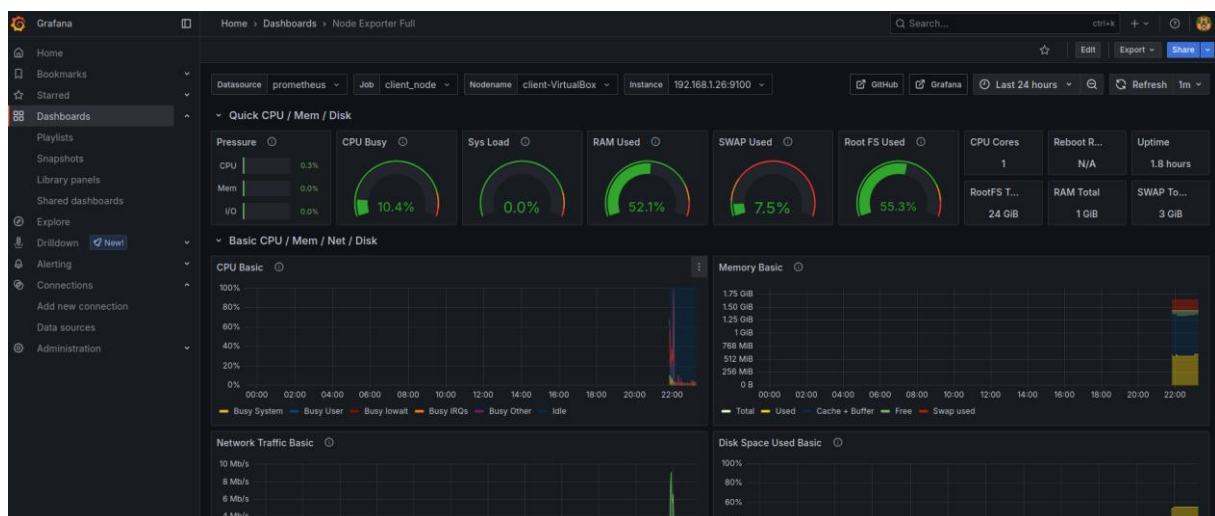
Folder

Unique Identifier (UID)
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
 [Change uid](#)

[Import](#) [Cancel](#)

6. Cliquer sur **Import**.

Le tableau de bord est maintenant disponible et permet de visualiser les données de la machine cliente Ubuntu (*client-virtualbox*).



 Analyse du Dashboard Grafana – Node Exporter Full :

Ce dashboard Grafana, basé sur Node Exporter et Prometheus, fournit une vue complète de l'état de la machine virtuelle (ici la machine cliente Ubuntu). Il permet de surveiller en temps réel les performances du système ainsi que l'utilisation des différentes ressources matérielles.

1. Vue d'ensemble (Quick CPU / Mem / Disk)

- **CPU Busy (10.4 %)** : taux d'occupation du processeur, ici modéré.
- **Sys Load (0.07)** : charge système moyenne très faible, signe d'une machine peu sollicitée.
- **RAM Used (52.1 %)** : environ la moitié de la mémoire est utilisée, ce qui reste acceptable.
- **SWAP Used (7.5 %)** : faible utilisation de la mémoire swap, à surveiller si elle augmente.
- **Root FS Used (55.3 %)** : plus de la moitié de l'espace disque racine est occupée.
- **CPU Cores : 1** : machine limitée à un seul cœur.
- **RAM Total : 1 GiB** : mémoire physique restreinte.
- **SWAP Total : 3 GiB** : espace swap plus important que la RAM, utilisé comme extension mémoire.
- **Uptime : 1.8 heures** : la machine est en fonctionnement depuis moins de 2 heures.

2. Détails des ressources

- **CPU Basic** : graphique montrant l'utilisation CPU (système, utilisateur, I/O, etc.). Ici, le CPU reste globalement sous-utilisé.
- **Memory Basic** : visualisation de la répartition mémoire (utilisée, cache, buffer, libre, swap). La mémoire est partiellement occupée avec une part importante de cache/buffer.
- **Network Traffic Basic** : suivi du trafic réseau entrant et sortant. Dans ce cas, le trafic est faible (0–6 Mib/s).
- **Disk Space Used Basic** : suivi de l'occupation disque. Le disque racine est rempli à environ 55 %.

3. Interprétation

- **Santé globale correcte** : faible charge CPU et mémoire dans des niveaux normaux.
- **Points de vigilance** :
 - L'utilisation du swap, indicateur potentiel de manque de mémoire.
 - L'occupation du disque, déjà à plus de 50 %.
- **Limitation matérielle** : machine avec peu de ressources (1 cœur CPU, 1 GiB RAM), ce qui peut devenir insuffisant selon la charge applicative.

Utilité du Dashboard

Ce dashboard permet :

- De contrôler en temps réel l'état du serveur.
- D'identifier rapidement les éventuels goulots d'étranglement (CPU, RAM, réseau, disque).

- De suivre l'évolution de la charge et d'anticiper des actions correctives (ajout de ressources, nettoyage disque, optimisation mémoire).

10. Conclusion

La mise en place d'une stack de supervision basée sur Prometheus, Node Exporter, OpenTelemetry Collector **et** Grafana permet de disposer d'une solution complète, flexible **et** extensible pour le suivi des systèmes et applications. grâce à cette architecture, il devient possible de :

- **Centraliser** la collecte des métriques provenant de serveurs Linux et Windows,
- **Analyser** les performances et détecter rapidement les anomalies,
- **Visualiser** l'état de l'infrastructure à travers des tableaux de bord dynamiques et personnalisables,
- **Standardiser** la gestion de la télémétrie via OpenTelemetry, facilitant l'interopérabilité avec d'autres outils.

Ce manuel fournit un guide pratique et reproductible pour déployer cette solution dans un environnement professionnel. Il constitue une base solide pour évoluer vers des pratiques plus avancées, telles que la supervision d'applications distribuées, l'intégration d'alertes ou la mise en place d'une infrastructure de supervision hautement disponible.